

Become Your Own Geofabrik: Self-Hosting OSM Regional Extracts as a Fan-Out-Native, Cost-Bearing Data Substrate

Abstract

1. Why we had to do it
2. Splitting the planet: extraction and its strategy trap
3. Making our own boundaries — and the assembly wall
4. Shapefiles, fallbacks, and region/level-aware providers
5. The single-atomic unit: a region is a relocatable job
6. Work effort inside the shape: measure the region, route the region
7. Fan-out: does it help, and how much
8. How exporting counties helped
9. Economics: is self-hosting ultimately faster than downloading from Geofabrik
10. Conclusion

Become Your Own Geofabrik: Self-Hosting OSM Regional Extracts as a Fan-Out-Native, Cost-Bearing Data Substrate

Research companion to the Facetwork thesis (`thesis.md`). Empirical basis: the July 2026 self-hosted-planet-split campaign — the `osm.planet` namespace, the `planet_bootstrap` / `boundary_gen` / `polygon_fetch` / `tiger_fetch` tools, and the fleet's execution records for the Canada, US, Mexico, German Länder/Kreise and US-county builds. All commits, handlers, FFL, and bucket inventories cited here are in the Facetwork repository (`fwh_osm`) and its MinIO object store.

Abstract

Geofabrik is the de-facto public source of per-region OpenStreetMap extracts. When it rate-limited and then IP-banned our fleet's shared egress, every OSM workflow that depended on a regional `.osm.pbf` broke at once — not because the pipeline was wrong, but because a third party we did not control had become a hard dependency on the critical path. We report on building a **drop-in Geofabrik replacement**: download the planet once, split it into a Geofabrik-compatible tree of per-region extracts, serve them from our own object store, and keep them current from OSM replication — behind a single environment variable (`FW_GEOFABRIK_BASE_URL`) so no consumer code changed.

The build was not a simple mirror. Making our *own* region boundaries — rather than depending on anyone's `.poly` files — turned into a multi-layered fight with the realities of OSM administrative geometry: `osmium export` silently drops boundaries it cannot assemble; the cheap extraction strategy clips a handful of edge nodes and thereby destroys the largest provinces; deep boundaries (counties) carry no ISO code and collide across states. We describe the five distinct failure modes we hit and the layered fallback that resolves each (`complete_ways` extraction; a region-**and**-level-aware ready-made poly provider — `osmfr` worldwide, US Census TIGER for the US; source-country keying; county-type-suffix normalization), and we argue the general lesson: robust admin-boundary assembly from raw OSM has a long tail, which is *why* dedicated boundary sources exist, and the pragmatic answer is a small hierarchy of fallbacks rather than a single clever algorithm.

We then make the paper's central architectural claim. Once each region is an independently extractable, self-contained unit of work, its **processing cost becomes measurable** — feature counts up front, peak resident memory empirically — and that cost signal is exactly what a distributed scheduler needs. We show a batcher that detects its own memory ceiling, sizes each osmium pass against a learned per-region cost, and self-heals on the OOM killer; we show the same self-contained unit surviving a mid-run host migration; and we show that fine-grained regions (US counties) let the whole thing **fan out** across the fleet, turning a serial multi-hour job into a parallel one whose wall-clock scales with runner count, not region count. Finally we address the economics directly: for a fleet that touches many regions repeatedly and needs them current, self-hosting is not merely a workaround for a ban — it is **faster and cheaper** than repeated public downloads, because it amortizes one planet fetch across cache reuse, incremental delta updates, and a granularity (per-county) the public source does not even offer.

1. Why we had to do it

Facetwork's OSM domain (`fwh_osm`) resolves a *region key* — `europa/france`, `north-america/us/california` — to a `.osm.pbf`, caches it, and runs handlers over it. The resolver's default remote was `https://download.geofabrik.de`. This worked until the fleet's egress IP accumulated enough automated requests that Geofabrik first throttled and then refused it. The failure was total and non-local: a downloader 404/403 propagated into every workflow, dead-lettered tasks fleet-wide, and no amount of retry helped because the refusal was by policy, not by transient error.

The first mitigation was a provider switch: `FW_OSM_EXTRACT_PROVIDER=osmfr` routes downloads to [OpenStreetMap France](#), which is not banned. This bought air, but it is the same class of dependency — a different third party on the critical path — and `osmfr` publishes only a French-centric *subset* of Geofabrik's regions (most US states, several countries, and all combined extracts are simply absent). The structural fix is to depend on **no external extract host at all**: become the source.

The design (Strategy A) is deliberately boring at the seam. Geofabrik's contract is two URL shapes — `<base>/<region>-latest.osm.pbf` for the extract and `<base>/<region>-updates/` for replication. If we publish exactly that layout to our object store and point `FW_GEOFABRIK_BASE_URL` at it, every existing consumer — the downloader, the delta-update path, the cache — keeps working unchanged. One environment variable is the entire integration surface. The hard part is everything upstream of that URL: producing the extracts, and producing the boundaries that define them.

2. Splitting the planet: extraction and its strategy trap

`planet_bootstrap` performs the split: one `osmium extract -c <config>` pass over the source (`planet-latest.osm.pbf`, 87 GB) writes every region's PBF, then each output is stamped with `our osmosis_replication_*` header so the delta path follows *our* server. The header indirection is what makes the replacement self-updating rather than a frozen snapshot: a region re-downloaded from us carries our replication URL, and `update-delta` applies OSM diffs against it.

The first non-obvious cost is memory, and it drove much of the campaign. `osmium extract` holds, per region in a pass, a node-id set over the planet's id-space; on a container whose Docker-Desktop VM has ~14 GiB (regardless of the host's 32–64 GB), packing too many regions into one pass invites the OOM killer (SIGKILL, exit `-9`). Canada at 25 regions/pass needed ~37 GB and died; a single dense province can approach the ceiling alone. This is the first appearance of the paper's theme: *the region is a unit of work with a real, and initially unknown, cost*, and a fixed `batch_size` is a guess. §6 replaces the guess with measurement.

The second cost is **strategy**, and it is subtler. `osmium extract` offers `simple`, `complete_ways`, and `smart`. `simple` is one pass and cheapest, but it *clips* ways at the extract boundary — it keeps a way that touches the region but drops the nodes that fall outside. For a country extract this is invisible in the data; it becomes catastrophic in §3.

3. Making our own boundaries — and the assembly wall

The interesting decision was to *generate* region boundaries from OSM itself rather than depend on anyone's `.poly` files. OSM carries `boundary=administrative` relations for essentially every admin unit on Earth (`admin_level` 2=country, 4=state/province, 6=county), so `boundary_gen` filters them at a level (`osmium tags-filter r/admin_level=N`) and assembles them to polygons (`osmium export -f geojsonseq --geometry-types=polygon`). This is self-contained, universal, and depends on no external boundary host. It also has a long tail of failure modes, which we hit one at a time — the empirical core of this paper.

Failure 1 — silent non-assembly. `osmium export` drops any boundary relation it cannot close into a valid polygon and reports *no error*. Running Canada at `admin_level=4` yielded only **7 of 13 provinces**; the six largest (Ontario, Quebec, BC, Nova Scotia, New Brunswick, Nunavut) were simply absent, with a clean "0 errors". Diagnosis required pulling the extract locally and dissecting the relations: Ontario had 442 member ways, *all present*, but **3 were node-incomplete** — three nodes clipped at the extract edge by the `simple` strategy of §2 broke the ring, and the whole province vanished. The most complex, most coastal boundaries are the most likely to lose an edge node, which is exactly why the survivors were the landlocked/island provinces.

Fix 1 — `complete_ways` from the source of truth. `complete_ways` keeps every node of a referenced way even outside the region, so boundaries are node-complete and assemble. This is also what Geofabrik itself does, so our extracts become genuinely self-contained (a real quality gain, not just a bug fix). Re-extracting Canada `complete_ways` from the *planet* (a continent extract is itself `simple` and clips its own edge — the North-America carve still lost Nunavut) recovered Ontario and BC: 7 → 9.

Failure 2 — the irreducible tail. Even `complete_ways` left four provinces out, for reasons `osmium export` cannot cheaply fix: Québec had one boundary way *outside the country poly* (a maritime segment), and Nova Scotia's boundary is a relation of **18 nested sub-relations** that `osmium export` will not recurse into. No extraction strategy closes these; they are limits of the assembler, not of the data.

Fix 2 — a ready-made-poly fallback for stragglers. For the regions self-generation cannot build, we fill from a provider that ships pre-assembled, robust `.poly` files, and we fill *only the gap* (`fetch_country_subregions` , compared by slug against what self-gen produced). This is the paper's practical thesis in miniature: **do not seek one algorithm that assembles every boundary on Earth; layer a cheap universal method under a robust ready-made source and take the union.** Canada reached all 13.

The general lesson is worth stating plainly: robust administrative-boundary assembly from raw OSM is a genuinely hard problem with a long tail — node clipping, maritime ways, nested relations, topology errors — and this is *precisely why Geofabrik and osmfr exist* as dedicated boundary sources. The self-hosting project does not eliminate that difficulty; it packages it behind a small fallback hierarchy so the common case is free and the tail is covered.

4. Shapefiles, fallbacks, and region/level-aware providers

The straggler provider is not universal, and neither of the two we use is — which forced the fallback to become **region- and level-aware**:

- **osmfr /polygons/** is worldwide (continents, ~199 countries, sub-regions for federal countries like Canada and Germany) but ships **no US state polygons** and no county tree.
- **US Census TIGER/Line** is authoritative for US states *and* counties but is **US-only**. TIGER ships ESRI **shapefiles**, so `tiger_fetch` downloads the `STATE / COUNTY` archives, reads them with `pyshp` , and writes one GeoJSON polygon per unit (`osmium extract` reads GeoJSON as readily as `.poly`).

So `fetch_country_subregions` dispatches by (*country, admin_level*): `US@4` → TIGER states, `US-state@6` → TIGER counties for that state, `everywhere-else@4` → osmfr, everything else → the empty set (no provider — which correctly stops a German county run from ever pulling in the 16 Länder). The shapefile route carried its own issues. Territories must be filtered (`STATEFP 60/66/69/72/78`) to match the 50-states-plus-DC set. And county names are the sharpest edge: Census `NAME` is the **bare** name (`Alachua`) while OSM names the same unit `Alachua County` , so the naive slugs (`alachua` vs `alachua-county`) do not match and the fallback publishes **both** — a silent 2× duplication we caught only by noticing Florida reported 129 "counties" against a true 67 (§7). The fix is a `_strip_admin_type` normalization (drop a trailing `County/Parish/Borough/Census-Area/Municipality`) so self-generated slugs equal TIGER's bare names and the fallback dedupes to one clean key.

A related keying failure recurred at every level: sub-country units were being keyed by an ISO→continent lookup (`boundary_gen` 's static map), which places Mexico under `central-america` even though the extract tree and `osmfr` both put it at `north-america/mexico`. Self-generated states landed under one prefix while the fallback's landed under another — 31 Mexican states hidden at `central-america/mexico/` while 3 sat at `north-america/mexico/`. The fix is to key sub-country units under the **source country from source_region**, which Facetwork already knows, rather than re-deriving a continent. The same fix simultaneously *enables counties*: German Kreise and US counties carry no ISO 3166-2, so an ISO-keyed filter dropped all 400 German counties as "noise"; keying under the known source country keeps them (`europa/germany/<kreis>`) and, for the US, the per-state fan-out yields **nested** `north-america/us/<state>/<county>` keys — essential, because ~30 states each contain a "Washington County" that a flat key would collide.

5. The single-atomic unit: a region is a relocatable job

An early distributed-execution bug shaped the rest. A multi-step FFL workflow (generate polys → extract → publish) handed **local file paths** between steps, but the fleet's scratch disk is per-host — the workflow broke the moment two steps landed on different machines. The resolution is `BuildAdminSet`: a **single-atomic** event facet that downloads its own source from the object store, generates, extracts, and publishes *all on the one host that claims the task*. No cross-host handoff; the region's whole pipeline is one relocatable unit of work.

This is more than a bug fix; it is the property that makes everything downstream possible. Because the unit is self-contained, it can be **scheduled, retried, migrated, and measured** as a black box. During one German-county run the task survived being killed on a smaller-VM host and re-claimed on a larger one — a live laptop→infra-host migration mid-job — precisely because it carried no host state. A self-contained region-job is the atom the rest of the system composes.

6. Work effort inside the shape: measure the region, route the region

Here is the paper's central contribution. Once a region is a self-contained job, the **effort to process it is a property of the shape**, and we can both *measure* it and *act* on it.

We measure it two ways. Up front, cheaply: `_feature_counts` gives node/way counts per region (already computed to skip empty regions). Empirically, precisely: the adaptive batcher wraps each `osmium extract` pass and polls the child's peak resident memory (`psutil`), so every pass reports the real cost of the regions it held.

We act on it in the batcher (`bootstrap_batched`, adaptive mode). It (1) detects the true memory **ceiling** — `cgroup memory.max`, else `/proc/meminfo MemTotal`, which is how it learns the Docker-VM's real ~14 GiB rather than the host's 32–64 GB; (2) sizes each pass under `ceiling × 0.7` from a learned per-region cost; (3) measures the pass's actual peak and updates the estimate (EWMA, persisted to a sidecar so the next run starts calibrated); and (4) **self-heals on OOM** — an osmium `-9` raises a recoverable error, the estimate is raised, and the *same* regions re-run in a smaller pass rather than dead-lettering. Empirically it detected 14.6 GB, sized down from 4 regions to 2 on the first OOM, and measured ~3.7 GB/region for `complete_ways` vs ~2.0 GB for `simple` — a real, learned distinction between strategies that no fixed constant would have captured.

The broader idea generalizes beyond memory. A shape that carries a *work-effort signal* is a shape a scheduler can route: heavy regions to capable hosts (the fleet's capability-tiered `server_groups`), cheap regions anywhere; expensive strategies where the RAM exists, cheap strategies where it does not. The region-poly is not merely a geometric mask — it is a **cost-bearing description of a routable job**, and treating it that way is what let a memory-bound fleet extract Ontario, 400 German counties, and thousands of US counties without a human guessing a batch size.

Two more resilience properties follow from treating the region as the unit. **Incremental publish**: each pass's extracts are uploaded immediately, so progress is durable. **Resume**: on entry a job lists what is already published under its prefix and skips it, so a large set *converges across retries* instead of restarting. The German counties proved it — 84 → 383 across five retries, an OOM series, a host migration, and MinIO load spikes, the count only ever rising.

7. Fan-out: does it help, and how much

Yes, and it is the clearest operational payoff of fine granularity. `BuildAdminSet` is single-atomic — one task, one host — which is correct but serial: ~3,143 US counties in one task run one after another. But counties **partition by state**, and each state's counties come from that state's already-published extract, so the natural fan-out unit is per-state. `BuildAdminFanout` is an FFL `foreach` over the direct-child extracts (`ListExtracts` enumerates the 51 states) that spawns **one `BuildAdminSet(admin_level=6)` task per state**, each distributed independently.

Observed live, the fan-out ran **8 states extracting counties concurrently across 8 distinct runner hosts**, so wall-clock $\approx$ the slowest single state rather than the sum of 51 — a runner-count-fold speedup, bounded by fleet size, not work size. Adding runners adds parallelism directly. This is the same shape as the domain's existing continent-heatmap fan-out (per-leaf across the fleet), and it is only possible because §5 made each region a relocatable job and §6 made each job right-sized for whatever host claims it.

Granularity and fan-out reinforce each other. Coarse regions (a continent) need a big box and run serially; fine regions (counties) are individually cheap and embarrassingly parallel. Self-hosting is what *creates* the fine granularity — Geofabrik does not publish per-county US extracts at all — so the fan-out is not merely enabled by self-hosting, it is a capability that only self-hosting offers.

8. How exporting counties helped

The US-county build was the campaign's hardest case, and it was worth doing precisely because it stressed every mechanism at once and left them stronger:

- It **forced the fan-out** to exist (3,143 units is intractable serially), producing `BuildAdminFanout` and `ListExtracts` — reusable for any "build one level below each child of a prefix" job.
- It **exposed the slug-collision and duplication bugs** ($30\times$ "Washington County"; `alachua` vs `alachua-county`) that coarser regions never triggered, yielding nested keying and `_strip_admin_type` normalization.
- It **validated the level-aware fallback** — TIGER counties are the only county provider for the US, so the county case is what proved the provider dispatch must key on `(country, admin_level)`, not country alone.
- It **demonstrated the resilience stack at scale** — adaptive batching, incremental publish, and resume all had to hold across dozens of parallel per-state jobs.

And the product itself is a capability the public source does not provide: a complete, delta-updatable, anonymously-downloadable tree of per-US-county extracts (`north-america/us/<state>/<county>`), keyed collision-free, that any downstream county-level analysis can pull exactly as it would a Geofabrik file. Exporting counties did not just add data; it hardened the whole substrate and extended it past parity with the thing it replaced.

9. Economics: is self-hosting ultimately faster than downloading from Geofabrik

For a one-off download of a single region, Geofabrik is obviously cheaper — a single GET. The comparison changes completely for a *fleet* that touches many regions, repeatedly, and needs them current, which is our case:

- **The ban made it non-optional**, but the benefits stand independently of it.
- **Amortized fetch.** Self-hosting pays one 87 GB planet download plus extraction compute, once; every subsequent regional read is a LAN GET from our MinIO with no external egress. A fleet re-reading regions across many workflows recovers that upfront cost quickly, and never pays a public host's rate limit again.
- **Incremental upkeep, not re-download.** The replication-header indirection means keeping current is applying OSM *diffs*, not re-fetching whole extracts — the marginal cost of freshness is proportional to change, not to size. This is the crux of "faster to upkeep ourselves": the public model is periodic full re-downloads; ours is deltas.
- **Cache locality.** The download path caches into `afl-cache` ; a region fetched once is reused fleet-wide with byte-for-byte integrity (verified sha256 on the US-state round-trip). Constant public downloads have none of this.
- **Granularity we choose.** We publish per-county extracts Geofabrik does not offer, at the exact partition the fan-out wants. The public source cannot be "faster" at serving a region it does not have.
- **No external coupling.** Only our own MongoDB and MinIO must be up; the OSM data plane has no third party on the critical path. (The one self-inflicted coupling we found and fixed — runner containers losing the infra IP after a DHCP drift — is now self-healed by the fleet-agent re-resolving the stable name into each container's `/etc/hosts` , so even our own infra address is not a brittle constant.)

The honest cost side: ~170 GiB of object storage for the extract tree, the compute for the initial planet split, and the engineering documented in §§2–7 — a real fallback hierarchy, an adaptive batcher, and a fan-out orchestrator. That engineering is the price of *robust* self-hosting; a naive mirror would have shipped the Ontario-shaped holes silently. Weighed against a hard dependency that can (and did) disappear by policy, and against a public model that re-downloads whole regions to stay fresh, self-hosting is not just a workaround — for a fleet at this usage it is the faster and more durable substrate.

10. Conclusion

We replaced an external, bannable, whole-region download service with an internal, delta-updated, fan-out-native one, behind a single environment variable. The build's real content was not the mirror but the boundaries: five distinct assembly failures resolved by a layered fallback (`complete_ways` extraction, a region/level-aware ready-made provider spanning `osmfr` and TIGER shapefiles, source-country keying, county-suffix normalization), whose general lesson is that raw-OSM boundary assembly has a long tail best packaged behind fallbacks rather than conquered by one algorithm. The architectural content is that a self-contained region-job carries a *measurable processing cost*, and a system that measures it can size, route, retry, migrate, and **fan out** that work automatically — which is what turned a memory-bound fleet into one that extracts thousands of counties in parallel without a human guessing a batch size. And the economic content is that, for a fleet that reads many regions and needs them current, owning the substrate — one planet fetch, cache reuse, incremental deltas, chosen granularity, no third party on the critical path — is ultimately faster to run and cheaper to keep current than the downloads it replaced.